Future Directions I

1. **Supply-chain provenance**: Integration with software supply chain frameworks such as in-toto [@torresarias2019intoto] and SLSA (Supply-chain Levels for Software Artifacts) [@openssf2023slsa] to provide end-to-end attestation from source commit through build pipeline to published artifact. SLSA's four graduated levels of build integrity (from basic provenance to hermetic, reproducible builds) provide a natural extension ladder for the template's currently document-centric provenance model. The template's existing steganographic layer embeds document-level provenance; supply-chain frameworks would add build-level provenance, closing the gap between "this PDF was produced by this pipeline" and "this pipeline was executed with this verified source code."
2. **Decentralized provenance**: Integration with IPFS or Arweave for immutable publication records, extending the SHA-256-based tamper detection to content-addressed storage networks.

Future Directions II

3. **Digital signatures**: GPG or X.509 signing integrated into the steganographic layer, providing cryptographic non-repudiation in addition to tamper detection.
4. **Continuous integration**: GitHub Actions workflows that execute the pipeline on every push, with PDF artifacts as release assets and automated DOI registration via Zenodo.
5. **Multi-language support**: Extension of the Thin Orchestrator pattern to R, Julia, and Rust projects, enabling polyglot research workflows within the Two-Layer Architecture.
6. **Automated FAIR4RS assessment**: Periodic self-scoring against FAIRsoft metrics [@garijo2024fairsoft], with quality indicators (executability, documentation completeness, metadata richness) tracked as pipeline artifacts alongside test coverage and rendering status.
7. **Knowledge graph integration**: Connecting pipeline outputs to Active Inference Knowledge Base entries for automated meta-analysis and cross-project citation tracking.

Future Directions III

8. **Formal verification**: Static analysis tooling to enforce the Thin Orchestrator pattern—verifying that scripts contain no algorithmic logic and that `src/` modules do not import from `scripts/`.

Future Directions IV

9. **Agentic research pipelines via MCP**: The `SKILL.md` descriptors already define the interface contracts for each infrastructure module; the natural next step is to expose them as MCP server endpoints [@anthropic2024mcp]. An MCP server wrapping `infrastructure.llm` would expose `query`, `review_manuscript`, and `translate_abstract` as protocol-native Tools; an MCP server wrapping `infrastructure.publishing` would expose `publish_to_zenodo` and `generate_citation_bibtex`. A research agent could then compose these Tools to execute the full pipeline—environment setup → test execution → analysis → rendering → validation → LLM review → DOI registration—without any human in the loop. This closes the loop opened by Lu et al.'s AI Scientist [@lu2024aiscientist], which demonstrated automated hypothesis generation and experimental iteration but relied on ad hoc laboratory scaffolding. `template/`'s pipeline, fully exposed as MCP tools, provides that scaffolding in a reproducible, versioned, and certified form. Longer-term, the Agent2Agent (A2A)

Conclusion I

`template/` demonstrates that high-integrity, reproducible research need not be onerous. By embedding provenance, testing, and documentation into the architecture itself—rather than layering them atop a fragmented workflow—the template transforms "best practices" from aspirational guidelines into enforced invariants [@wilson2017good;@sandve2013ten;@lamprecht2020towards]. The Two-Layer Architecture ensures that infrastructure improvements propagate to all projects without coupling. The Zero-Mock policy ensures that tests reflect reality. The steganographic provenance layer ensures that published artifacts carry their own authentication. The comparative analysis confirms that no existing tool integrates all eleven distinctive capabilities—testing enforcement, coverage thresholds, cryptographic provenance, steganographic watermarking, multi-project management, AI-agent documentation, the agentic skill protocol, interactive TUI, Zero-Mock policy, manuscript rendering, and pipeline orchestration—within a single enforced pipeline.

Conclusion II

The template is not merely a build tool; it is an epistemological commitment. It asserts that a research paper is not a static document but a build artifact—reproducible, verifiable, and traceable to the code that generated it. As Knuth observed, programs should be written for humans to read and only incidentally for machines to execute [@knuth1984literate]. We extend this dictum: research manuscripts should be *built* for verification and only incidentally for reading. In an era of generative AI, AI-native research workspaces, and synthetic media—where the boundary between human-authored and machine-generated text grows increasingly indeterminate [@gruenpeter2021research]—the provenance chain from source code to published PDF is not an administrative convenience. It is the epistemic ground on which scientific trust must be rebuilt. That this manuscript was itself built, tested, and watermarked by the pipeline it describes—its metrics computed from the repository it inhabits, its figures rendered by the code it documents—is not a rhetorical device but a structural proof: the system works because you are reading its output.